

Day 50: Product Class

Day 50: Product Class

1. Day Number

Day 50

2. Topic Name

Product Class

Today you will learn how to represent a product using a Python **class** instead of a dictionary.

3. Connection

Earlier, you created product information using dictionaries, for example:

```
product = {  
    "name": "Laptop",  
    "price": 50000,  
    "stock": 5  
}
```

Today, you will represent the same type of information using a **Product class**.

Conceptually:

```
Dictionary approach  
  ↓  
{  
    name,  
    price,  
    stock  
}  
  
    becomes  
  
OOP approach  
  ↓  
Product class  
  ├── name  
  ├── price  
  └── stock
```

This continues the OOP concepts you practiced with User and BankAccount.

4. Important Topics

Today, focus on three important concepts:

- class
- constructor `__init__()`

- attributes

The basic structure is:

```
class Something:
    def __init__(self, value1, value2):
        self.value1 = value1
        self.value2 = value2
```

5. Foundational Notes

What is a class?

A **class** is a blueprint for creating objects.

For example:

```
Product
```

can describe what every product should contain.

A product may have:

```
name
price
stock
```

So the class acts like this blueprint:

```
Product
|
├─ name
├─ price
└─ stock
```

What is an object?

An **object** is an actual item created from a class.

For example:

```
Product class
  ↓
product1
  ↓
Laptop
50000
5
```

Product is the blueprint.

product1 is the actual object.

What is __init__()?

__init__() is the constructor.

It runs automatically when you create an object.

Example structure:

```
class Student:
    def __init__(self, name):
        self.name = name
```

If you create:

```
student1 = Student("Amit")
```

Python automatically calls the constructor.

Conceptually:

```
Student("Amit")
  ↓
__init__()
  ↓
self.name = "Amit"
```

What is self?

self represents the **current object**.

For example:

```
self.name = name
```

means:

| Store the received name inside this particular object's name attribute.

If:

```
name = "Laptop"
```

then:

```
self.name
```

will store:

```
"Laptop"
```

What are attributes?

Attributes are variables that belong to an object.

For a product:

```
product1
├── name  → "Laptop"
├── price → 50000
└── stock → 5
```

You can access them using dot notation:

```
object.attribute
```

For example:

```
student1.name
```

6. Easy Example

Let's use a different class so that we do not solve today's problem directly.

Suppose we want to represent a book.

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages
```

Create an object:

```
book1 = Book("Python Basics", 200)
```

Access the attributes:

```
print(book1.title)
print(book1.pages)
```

Output:

```
Python Basics
200
```

Here:

```
Book
├── title
└── pages

    ↓ create object

book1
├──
```

```
title = "Python Basics"
pages = 200
```

The same idea can be applied to a product.

7. Problem Statement

Create a Product class with three attributes:

```
name
price
stock
```

Use a constructor to initialize these values.

Then:

1. Create **one product object**.
2. Give it a product name.
3. Give it a price.
4. Give it a stock quantity.
5. Print the product details using the object's attributes.

Keep today's program simple.

Do not add methods such as `display()`, `buy()`, or `update_stock()` yet.

The purpose today is to practice:

```
class → constructor → attributes → object
```

8. Concepts Used

You will use:

Class

```
class Product:
```

Defines the blueprint.

Constructor

```
def __init__(...):
```

Initializes the object.

self

Represents the current product object.

Attributes

You need three attributes:

```
name
price
stock
```

Object creation

You will create one object from the Product class.

Dot notation

You will access values using:

```
object.attribute
```

9. Thought Process

Before coding, think about the problem in small steps.

Step 1: What object am I representing?

A product.

So the class should be:

```
Product
```

Step 2: What information does a product need?

The problem gives three pieces of information:

```
name
price
stock
```

Step 3: Where should these values be initialized?

Inside the constructor:

```
__init__()
```

Step 4: What should the constructor receive?

It needs:

```
name
price
stock
```

plus:

```
self
```

Step 5: Where should the values be stored?

Inside object attributes.

Think:

```
incoming name
  ↓
self.name

incoming price
  ↓
self.price

incoming stock
  ↓
self.stock
```

Step 6: Create one object

Conceptually:

```
Product
  ↓
create
  ↓
product1
```

Step 7: Print the attributes

Use dot notation:

```
product1.name
product1.price
product1.stock
```

10. Pseudocode

```
START

Create Product class

    Create constructor that receives:
        name
        price
        stock

    Store name in object
    Store price in object
    Store stock in object
```

```
Create one Product object
Give it a name
Give it a price
Give it a stock value

Print product name
Print product price
Print product stock

END
```

Visual flow:

```
START
|
v
Define Product class
|
v
Define __init__()
|
+----- name
|
+----- price
|
+----- stock
|
v
Create product object
|
v
Print attributes
|
v
END
```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Your program should roughly have two sections:

1. Class definition
2. Object creation and printing

Structure:

```
Product class
|
└─ constructor
    |
    └─ name
```



For today's exercise, avoid adding unnecessary logic.

You do **not** need:

```
loops
lists
dictionaries
methods other than __init__()
file handling
inheritance
```

12. Easy Edge Cases

Edge Case 1: Stock is 0

Example conceptually:

```
name = "Laptop"
price = 50000
stock = 0
```

This can represent a product that is currently out of stock.

Your class should still be able to store it.

```
Product
├── name = Laptop
├── price = 50000
└── stock = 0
```

No special validation is required today.

Edge Case 2: Price is 0

Example:

```
name = "Sample"
price = 0
stock = 5
```

A zero price should still be stored correctly for this exercise.

You are currently practicing **object creation**, not price validation.

Later you could add validation such as:

```
price must not be negative
```

but don't complicate today's exercise with that yet.

13. Expected Input

You do not necessarily need `input()` today.

You can directly provide values when creating the object.

For example, your test data could conceptually be:

```
Name: Laptop  
Price: 50000  
Stock: 5
```

These values should be passed to the constructor when the object is created.

14. Expected Output

Your output could look similar to:

```
Product Name: Laptop  
Price: 50000  
Stock: 5
```

The exact formatting is not important.

The important part is that the values come from the object's attributes:

```
product object  
  |  
  +---- name  -----> printed  
  |  
  +---- price -----> printed  
  |  
  +---- stock -----> printed
```

15. Hint Only

Start with:

```
class Product:
```

Then think about what parameters the constructor needs:

```
def __init__(self, ?, ?, ?):
```

You need three attributes:

```
self.?  
self.?  
self.?
```

After completing the class, create one object:

```
product1 = Product(...)
```

Finally, access its information using:

```
product1.?
```

Your target structure

```
Product class  
  |  
  v  
__init__()  
/   |   \  
name price stock  
  |  
  v  
Create product1  
  |  
  v  
Print its attributes
```

Main goal for Day 50: Be comfortable converting a simple product dictionary idea into a class containing constructor-based attributes.